

LenderBook Ramaiah Finance

LenderBook is a premium, full-featured lender management dashboard designed to track given loans, borrowers, repayments, borrowings (loans taken), and daily route expenses. It is optimized for use in local village finance businesses with offline-first capabilities (caching data in `localStorage` and syncing with Supabase when connected), and includes a native Android wrapper app that intercepts banking SMS alerts to automatically reconcile payments.

Architecture Overview

The system consists of two primary components:

1. **Web Dashboard (PWA)**:
 - * Built with HTML, Vanilla CSS, and JavaScript.
 - * Uses **Vite** for fast compiling and building.
 - * Integrated with **Supabase** for cloud storage, user accounts, and real-time database sync.
 - * Stores records locally in `localStorage` to allow 100% functional offline operations, which automatically syncs back when a network connection is detected.
 2. **Native Android Wrapper (`lenderbook-android`)**:
 - * Built with Kotlin, Jetpack Compose, and Gradle.
 - * Wraps the live hosted dashboard (`https://finance-beta-three.vercel.app`) in a fullscreen WebView container.
 - * Requests native Android permissions (`RECEIVE\_SMS` and `READ\_SMS`) to run an SMS BroadcastReceiver that intercepts credit notifications from banks and forwards them directly to the web app interface.
-

Android Application Details

The Android application folder is located at `/lenderbook-android`.

1. WebView Settings

In order for the web dashboard to function properly within the native container, the WebView in `MainActivity.kt` is configured with:

- **JavaScript Enabled**: Necessary for rendering the dynamic web page and executing client-side scripts.
- **DOM Storage Enabled**: Crucial for keeping Supabase authentication tokens and client-side database caches active.
- **Database Caching Enabled**: Supports indexed database storage on the device.

2. Native System Intents Bridge (Dialer, SMS, WhatsApp)

Since standard WebViews do not support custom URI schemes out of the box, `MainActivity.kt` intercepts the following link clicks in the web application and launches native system intents:

- **Dialer (`tel:+91...`)**: Launches the native phone dialer with the borrower's number.
- **System SMS (`sms:+91...`)**: Launches the native messaging app pre-filled with the reminder message.
- **WhatsApp (`whatsapp://...` or `https://wa.me/...`)**: Launches the WhatsApp application to share PDF Passbook ledgers directly.

Native SMS Interception & JS Bridge

To automate payment tracking, the Android app reads incoming banking SMS alerts and passes them directly to the web dashboard.

A. Manifest Permissions

The app requests these permissions inside `AndroidManifest.xml`:

```
<!-- Allows loading the hosted Vercel URL -->
<uses-permission android:name="android.permission.INTERNET" />
<!-- Allows listening to incoming SMS events -->
<uses-permission android:name="android.permission.RECEIVE_SMS" />
<!-- Allows reading message details (sender header and message text) -->
<uses-permission android:name="android.permission.READ_SMS" />
```

B. SMS Interceptor Flow

1. Upon start, the app prompts the user to grant **SMS Permissions** using Android runtime dialogs.
2. A dynamic `BroadcastReceiver` is registered to listen to `SMS\_RECEIVED\_ACTION`.
3. When a new text message arrives, the receiver captures the sender (e.g. `VK-HDFC`, `AX-SBI`) and the message text.
4. The receiver escapes any quotation marks and line breaks to prevent script injection.
5. It invokes the JavaScript bridge in the WebView:

```
```javascript
window.handleAndroidIncomingSMS(sender, body);
```
```

C. Web Application Hook

Inside src/main.js, the global bridge handler parses and processes the transaction:

```
window.handleAndroidIncomingSMS = async function(sender, body) {
  try {
    // 1. Logs raw SMS in offline/online cache
    const newSms = await addRawSms({ smsText: body, sender: sender });
    // 2. Automatically parses the SMS (identifies transaction amount & matches borrower via UPI/VPA)
    await processIncomingSmsRow(newSms);
    // 3. Syncs data and updates the UI dashboard with toast alerts
    await refreshData();
    renderPage(currentPage);
  } catch (err) {
    console.error("Error in handleAndroidIncomingSMS:", err);
  }
};
```

Building & Installing the App

Prerequisites

- **Java Development Kit (JDK)**: JDK 17 is required. We recommend installing `Microsoft OpenJDK 17` or `Eclipse Temurin 17`.
- **Android SDK**: Build Tools 36 and Platform Tools 37.

Building the APK

1. Set your `JAVA\_HOME` environment variable to point to your JDK installation.
2. Navigate to the `lenderbook-android` directory.
3. Run the Gradle build script to compile the project:

```
``powershell
# Windows PowerShell
$env:JAVA_HOME = "C:\Program Files\Microsoft\jdk-17.0.19.10-hotspot"
$env:Path = "$env:JAVA_HOME\bin;" + $env:Path
cmd.exe /c ".\gradlew.bat assembleDebug"
``
```

4. Once completed, the debug APK will be generated at:
`lenderbook-android/app/build/outputs/apk/debug/app-debug.apk`

Installing on a Phone

For convenience, a compiled build is copied directly to the root project folder:

- **Pre-compiled APK**: LenderBook.apk (approx. 11.9 MB)

Steps to install:

1. Copy the `LenderBook.apk` file to your Android phone (via USB, email, or WhatsApp).
2. Open the file on your phone using a File Manager app.
3. If prompted, enable **"Install from Unknown Sources"** in your phone settings.
4. Install the application.
5. Open **LenderBook** and tap **"Allow"** on the prompt requesting permission to receive and read SMS messages.